Quantize for the GPU, Not the Neural Engine: Optimizing On-Device LLM Decode for Apple Core ML on a 16 GB Mac

Ali Asaria
Transformer Lab

Tony Salomone
Transformer Lab

Deep Gandhi*
Transformer Lab

Abstract

Running a language model on an Apple device through Core ML offers two tempting levers for speed: the Apple Neural Engine, a dedicated accelerator, and low-bit weight compression such as palettization, which Apple's tooling exposes natively. Conventional wisdom treats both as the obvious path to fast on-device inference. We test that wisdom directly. Converting a dense 1.7B-parameter language model to Core ML and benchmarking it on a 16 GB M3 MacBook Air, we sweep post-training quantization schemes against the three Core ML compute-unit policies and measure decode throughput, model size, and task quality. The result inverts the common assumptions: within Core ML, plain int4 *linear* weight quantization run on the *GPU* decodes 3.2× faster than the fp16 Core ML baseline while shrinking the model 3.3× and holding task accuracy within measurement noise, whereas a default conversion is never placed on the Neural Engine to any benefit (its requested-ANE decode is indistinguishable from CPU-only, i.e. it falls back to the CPU) and Apple-native palettization at 4 bits is actually slower than fp16 on the GPU. These outcomes are consistent with single-token decode being memory-bandwidth bound and with how each backend handles low-bit weights, though we did not instrument that mechanism directly. The practical recipe is simple and training-free; the negative results, that a default conversion will not use the Neural Engine and that palettization does not pay off on the GPU, are, we argue, the more useful contribution.

1 Introduction

On-device language models promise privacy and zero network latency, but a phone or laptop constrains a model to its unified memory and to whichever compute units the runtime exposes. On Apple silicon, Core ML routes work across three of them: the CPU, the GPU, and the Apple Neural Engine (ANE). Two ideas dominate practitioner intuition about making such models fast. First, the ANE is a purpose-built neural accelerator, so it "should" be the target. Second, Apple's `coremltools` exposes *palettization* (k-means weight clustering) as a first-class, ANE-resident compression scheme, so it "should" be the way to shrink and speed up a model. Both are sensible defaults, and Apple's tooling actively encourages them, yet on a memory-constrained device they are rarely measured end to end against the plainer alternatives.

We measure them. Starting from a dense, open-weight 1.7B-parameter instruction model, we build a reproducible Core ML conversion pipeline, then run a controlled sweep over post-training

*Corresponding author: `deep@lab.cloud`

weight-quantization schemes (int8 and int4 linear quantization, and 3- and 4-bit palettization) crossed with the three Core ML compute-unit policies, all on a single 16 GB M3 MacBook Air. We report decode throughput, prefill latency, on-disk size, and a held-out task-accuracy check. Throughout, "decode throughput" is measured on a fixed-shape probe and the comparison is internal to Core ML; we state these scopes carefully (§4) because they bound what the negative results can claim.

Our findings, for a default Core ML conversion on this device, are:

1. **A default conversion does not use the Neural Engine.** Requesting the ANE yields decode latency indistinguishable from CPU-only (within the run-to-run spread), i.e. the graph falls back to the CPU; the GPU is significantly faster than the requested-ANE path (a gap far larger than the spread), and Core ML's automatic policy itself routes to the GPU. We did not hand-optimize an ANE-specific graph, so this bounds default conversions, not the ANE's ceiling (§5).

2. **Int4 *linear* quantization on the GPU is the most effective scheme.** It decodes 3.2× faster than the fp16 Core ML baseline at 3.3× smaller size with task accuracy within measurement noise (§5).

3. **Apple-native 4-bit palettization is slower than fp16 on the GPU.** The measurement is clear; the cause is not isolated (the 3-bit cell is faster than the 4-bit one, which complicates a simple lookup-table explanation), and the int8 and palettization cells were incoherent under a configuration that also quantizes the tied embedding (§6).

4. **An open, training-free conversion-and-benchmark pipeline** and the documented negative results, so a practitioner can reproduce the recipe and skip the ANE/palettization detour (§4, §6).

2 Related Work

LLM inference on Apple silicon. Most recent work on running language models on Apple hardware targets the GPU through MLX or Metal rather than Core ML and the ANE. Barrios [1] build a native Apple-silicon engine for text and multimodal models, and Vegasena [2] implement fused int4 key/value-cache attention in custom Metal kernels; both run on large-memory Macs and on the GPU. Ajayi and Odunayo [3] benchmark on-device ML on Apple silicon with MLX, and Hendria [4] document non-monotonic decode latency under Apple's MPS backend, underscoring the need to report latency distributions rather than single points. Work that does target the ANE is narrow: Benazir and Lin [5] accelerate mixture-of-experts *prefill* on the ANE but note that single-token decode is memory-bound and does not amortize CPU–ANE coordination, and that the ANE dequantizes low-bit weights to FP16 before computing. Ochiai [6] find, for diffusion models on a large Mac, that Core ML conversion is the only reliable win while quantization and parallelism give little benefit. To our knowledge, ours is the first study to directly compare the ANE, GPU, and CPU for dense small-LLM *decode* under Core ML on a memory-constrained device.

Weight quantization and palettization. Post-training quantization is a mature lever for shrinking language models [7]. Song and Lin [8] recover low-bit accuracy without GPUs via k-means layer splitting, Liu et al. [9] push extreme low-bit weight *clustering* (the same primitive as Core ML

palettization), and Lee et al. [10] search layer-wise mixed precision to meet a memory budget. These motivate our palettization and int4 cells; on the Apple GPU we find the linear int4 representation markedly outperforms the palettized cells for decode, though our sweep varies representation and granularity together and so does not isolate bit-width as the cause (§7).

Pruning, offloading, and on-device VLMs. Rath and Maliakkal [11] show that unstructured pruning yields no speedup on Apple silicon, which led us to exclude pruning. Flash-to-DRAM offloading schemes [12, 13] and edge–cloud split inference [14] target a different design point than our fully on-device, unified-memory setting. On-device vision-language deployment [15, 16] informs a planned extension but is outside this study's scope.

3 Method

Conversion pipeline. We convert a dense decoder model from its PyTorch definition to a Core ML ML Program (FP16 activations) with `coremltools`. Two practical pins were required for a modern model to convert at all: the quantization-cast path in current `coremltools` assumes a NumPy 1.x scalar conversion, and recent model code emits ops the converter cannot lower, so we pin a conversion-compatible toolchain (§4) and trace through TorchScript. We build two converted forms: a fixed-length stateless model used for output-correctness and task-accuracy checks, and a stateful fixed-shape key/value-cache model used as a single-token decode-latency probe. The probe traces a fixed cache position, so it measures the per-step weight-streaming cost of decode at short context rather than full autoregressive generation; correctness and latency are therefore measured on different artifacts, and we treat the latency numbers as a per-step microbenchmark (§4, §7).

Optimization cells. On the converted fp16 model we apply, post-training and without calibration data, four weight-only schemes via `coremltools.optimize.coreml`: int8 and int4 linear (symmetric) quantization, and 4-bit and 3-bit k-means palettization. int4 linear uses per-block granularity (block size 32).

Placement. Each model is run under the three Core ML compute-unit policies (all units, CPU+GPU, and CPU+ANE), plus CPU-only as a floor, so that the effect of the accelerator is isolated from the effect of the weight representation. Prefill and decode are timed separately, since they have different compute profiles.

4 Experimental Setup

Model and device. The base model is SmolLM2-1.7B-Instruct, a dense Llama-architecture decoder (24 layers, hidden size 2048, 32 attention heads, vocabulary 49,152, tied input/output embeddings, ≈1.71B parameters) with the standard $1/\sqrt{d}$ attention scaling. All measurements are taken on a single 16 GB Apple M3 MacBook Air.

Quality. Task accuracy is measured on a frozen subset of 120 four-choice ARC-Easy questions, scored by greedy letter choice on the converted Core ML artifacts (a longer-context, full-logits variant so the answer position is read without padding contamination). The frozen fp16 conversion

Table 1: fp16 SmolLM2-1.7B on Core ML, by compute-unit policy (16 GB M3 Air). Decode is a single-token per-step microbenchmark (15 timed runs); prefill is a 64-token forward pass (7 runs); IQR in parentheses. The GPU is faster than the requested-ANE path by far more than the spread, while the requested-ANE path is indistinguishable from CPU-only (CPU fallback). For external context, MLX fp16 on the same device decodes at 25.6 tok/s.

Compute units	Decode (tok/s)	Decode ms (IQR)	Prefill ms (IQR)
All (routes to GPU)	23.8	42.1 (1.1)	98.4 (0.7)
CPU + GPU	23.4	42.7 (2.8)	98.1 (0.8)
CPU + ANE	20.7	48.4 (0.8)	109.8 (0.4)
CPU only	20.7	48.2 (1.0)	111.4 (0.7)

reproduces the reference PyTorch model's next-token prediction exactly (logit cosine 1.0, identical argmax), so quality differences are attributable to quantization.

Latency. Decode throughput is measured on the stateful key/value-cache probe (a fixed-position per-step microbenchmark, see §3); prefill on a fixed 64-token input. We discard warm-up runs and report the median over timed runs (15 for decode, 7 for prefill), with the inter-quartile range (IQR) to quantify the run-to-run spread, since decode latency on Apple silicon is non-monotonic and single-point timings mislead. Decoding is greedy, so no sampling seed is needed; the quantization is deterministic. Quality is the held-out accuracy of §4; with 120 items the test resolves only differences of roughly ten points, which bounds what the accuracy comparison can establish (§7).

Toolchain. `coremltools` 9.0, PyTorch 2.7.1, Transformers 4.46.3, and NumPy 1.26.4. Pins, model revision, data hashes, and exact commands are in the reproducibility package.

5 Results

A default conversion does not engage the Neural Engine. Table 1 reports the fp16 model under each compute-unit policy, with the IQR over timed runs. Two facts stand out. First, the GPU is faster than the requested-ANE path by a margin far larger than the run-to-run spread (decode 42.1 vs. 48.4 ms, IQR ≈1 ms; prefill 98.4 vs. 109.8 ms). Second, the requested-ANE path is statistically indistinguishable from CPU-only (decode 48.4 vs. 48.2 ms), which is the signature of a graph that fell back to the CPU rather than running on the accelerator. Core ML's automatic "all units" policy itself selects the GPU. We could not get a default conversion to benefit from the ANE; this is a statement about default conversions, not about the accelerator's ceiling, since we did not build an ANE-specific graph (§7).

Int4 linear quantization is the most effective scheme. Table 2 sweeps the four post-training schemes on the GPU. int4 linear quantization decodes at 75.8 tok/s, a 3.2× speedup over the 23.8 tok/s fp16 baseline, while shrinking the model from 3.2 GB to 0.96 GB (3.3×). 4-bit palettization, despite a comparable size, is *slower* than fp16 (10.8 tok/s); curiously the 3-bit cell is faster than the 4-bit one (25.1 vs. 10.8 tok/s), an inversion we cannot explain and that complicates any simple account of the slowdown (§6). Among the cells, only int4 linear preserved the model's output: on

Table 2: Post-training weight-quantization cells on the GPU. Decode speedup is relative to the 23.8 tok/s fp16 baseline. "Coherent" is whether greedy generation remained fluent; incoherent cells are reported but not released.

Scheme	Size (GB)	Decode (tok/s)	vs. fp16	Coherent
fp16 (baseline)	3.2	23.8	1.0×	yes
int4 linear (per-block 32)	0.96	75.8	3.2×	yes
int8 linear (per-channel)	1.71	46.0	1.9×	no
palettize 4-bit (k-means)	0.86	10.8	0.45×	no
palettize 3-bit (k-means)	0.64	25.1	1.1×	no

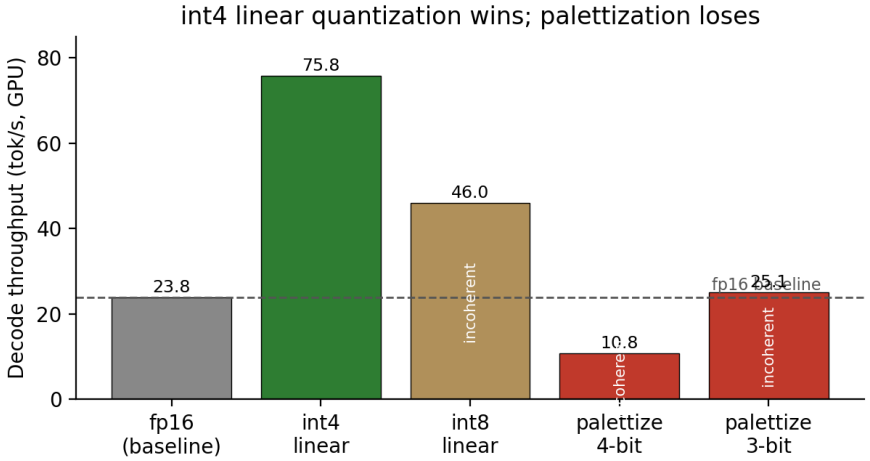

Figure 1: Decode throughput on the GPU by weight-quantization scheme. int4 linear quantization is the only cell that is both fast and coherent; the int8 and palettization cells are reported for completeness but produce incoherent output as configured (§6).

the stateless correctness model its logits match fp16 at cosine 0.948 with identical greedy argmax, and on a set of open-ended prompts its greedy generations remain fluent and on-topic. The int8 and palettization cells, as configured, produced incoherent output. We caution that these cells vary representation *and* granularity at once and all quantize the tied embedding, so this table compares four *configurations*, not four bit-widths; the controls needed to attribute the incoherence to bit-width are not yet run (§7).

Quality is not degraded at the resolution we can measure. On the 120-question ARC-Easy subset, the int4 model answers 88/120 correct (73.3%) against the fp16 model's 87/120 (72.5%; Table 3). The difference is a single question; on 120 items the 95% confidence interval on the accuracy difference is roughly ±10 percentage points, so the test resolves only large differences. It establishes that int4 does not regress at the ten-point scale, and is equally consistent with small gains or small losses; it cannot, on its own, confirm the $\leq 2\%$ ($\approx$1.5-point) guardrail, only that the guardrail is not contradicted. We therefore claim no measurable quality change for int4, not parity in a strong statistical sense. With the 3.2× decode speedup, int4 satisfies the speed and memory criteria outright and is consistent with the quality criterion ($\geq$ 1.5–2× faster at $\leq 2\%$ drop, within

Table 3: Held-out task accuracy, ARC-Easy, 120 four-choice questions, greedy letter-choice. The int4 delta is within evaluation noise.

Model	ARC-Easy accuracy
fp16	72.5%
int4 linear	73.3%

16 GB), pending a larger evaluation.

6 Discussion

What the compute-unit result does and does not show. The requested-ANE path matched CPU-only to within the run-to-run spread (Table 1), which is the behavior of a graph that ran on the CPU rather than the accelerator; we read the result as "a default conversion is not placed on the ANE," not "the ANE is intrinsically unhelpful." Why a default conversion avoids the ANE is plausibly tied to the ANE's preference for specific fixed-shape, FP16 layouts and to its dequantizing low-bit weights to FP16 before computing, as Benazir and Lin [5] note, so quantization would save bandwidth but not ANE arithmetic; we did not instrument this, and an ANE-targeted graph might engage the accelerator. What the data do support directly is the practical recommendation: for a default conversion, target the GPU, and expect the automatic policy to do so. We did not instrument a roofline, so the further claim that decode is bandwidth-bound is the standard expectation for single-token generation rather than a measurement of this model.

The palettization slowdown is real but unexplained. Palettization stores weights as codebook indices, so a matrix multiply must gather values through a lookup table, and one might expect that gather to cost more than streaming a linearly-quantized tensor. But our own numbers undercut a simple gather-cost story: the 3-bit cell (25.1 tok/s) is more than twice as fast as the 4-bit cell (10.8 tok/s) and faster than fp16, whereas a gather-bound account would predict similar costs. We therefore do not claim a mechanism; the 4-bit cell most likely hit an implementation or fallback pathology we did not isolate. The defensible, measured statement is narrow: in this setting, the Apple-native palettization cells did not beat int4 linear for GPU decode, and the 4-bit cell was slower than fp16.

Why int8 and palettization were incoherent. All four cells quantize every weight, including the model's tied input-embedding/output-projection matrix, which maps directly to the logits. The cells differ in granularity: int4 uses fine per-block (32) scales, int8 uses coarser per-channel scales, and palettization replaces weights with codebook indices. We did not isolate the cause, but the leading hypothesis is that int4's fine-grained scales absorb the error on the sensitive shared matrix while the coarser int8 and the clustered palettization do not. If so, this is a configuration failure rather than a property of int8 or of low-bit quantization in general, and excluding the embedding/output projection from quantization is the natural test; we have not yet run it. We report these cells as honest negative results rather than tuning them away.

7 Limitations

This study validates one model on one device against one task, so external validity is limited, and several internal caveats bound the claims. **Measurement.** Decode latency comes from a stateful probe with a trace-fixed cache position, so it measures per-step weight-streaming at short context rather than full autoregressive generation, and it is a different artifact from the stateless model used for correctness and accuracy; very long contexts, where the key/value cache grows large, are not stress-tested. Latency is reported as a median with IQR but without a formal test, and the M3 Air is fanless, so thermal state across conditions is uncontrolled; the small within-policy gaps (for example All vs. CPU+GPU) should be read as indistinguishable. **Compute units.** Our result is that a *default* conversion is not placed on the ANE (the requested-ANE path matches CPU-only), not that the ANE cannot help; we did not build an ANE-targeted graph, and the quantized-on-ANE arm could not be run at all because the quantized stateful model failed to load on the ANE. **Confounds.** The quantization cells vary representation and granularity together and all quantize the tied embedding, so the table compares configurations rather than bit-widths; the controls that would attribute the int8 and palettization incoherence to bit-width (int8 per-block, int4 per-channel, embedding-excluded variants) were not run. **Statistical power.** The 120-item accuracy test resolves only ≈10-point differences, so it cannot confirm the $\leq 2\%$ guardrail, only that it is not contradicted; the incoherent cells have no accuracy numbers and are dismissed by logit cosine alone. **Baseline.** The 3.2× speedup is internal to Core ML; we report an MLX fp16 reference (25.6 tok/s) but did not benchmark an MLX or Metal int4 path, so we cannot claim the Core ML int4 model is competitive with the dominant practical on-device backends. **Scale.** The 7B-parameter scale-up could not be produced on the 16 GB device: converting a 7B model exceeds its memory and disk at conversion time, even though the resulting int4 *runtime* artifact (≈4 GB) would fit; this awaits a larger machine. **Quality breadth.** Calibration and subgroup-bias evaluation remain to be done, and compression is known to affect bias; models with non-standard attention scaling (for example a scale of 1.0) are untested.

8 Availability

The exact toolchain pins, base-model revision, data hashes, and the commands that produce every number are documented so the recipe can be reproduced; the conversion and benchmark code, the converted model artifacts, and the evaluation data are available from the authors on request.

9 Conclusion and Future Work

For dense small language-model decode on a 16 GB Apple M3 MacBook Air under Core ML, the effective optimization is int4 linear weight quantization executed on the GPU: 3.2× faster decode and 3.3× smaller, with task accuracy unchanged at the resolution we could measure. A default conversion never engaged the Apple Neural Engine, the accelerator one would reach for first (its requested-ANE runs fell back to the CPU), and Apple's native palettization was no faster than fp16 on the GPU. The practical takeaway for a developer is compact: convert to Core ML, apply int4 per-block linear weight quantization, run on the GPU, and do not expect a default graph to use the Neural Engine. The most useful next steps are to test whether excluding the tied embedding rescues the int8 and palettization cells, to add an ANE-targeted graph and an external (MLX/Metal) int4

baseline, to reproduce the recipe on a larger machine for 7B-class models, and to broaden the quality and robustness evaluation beyond a single task.

References

[1] Wayner Barrios. Native LLM and MLLM Inference at Scale on Apple Silicon. 2026. URL https://arxiv.org/abs/2601.19139.

[2] Sai Vegasena. Open-TQ-Metal: Fused Compressed-Domain Attention for Long-Context LLM Inference on Apple Silicon. 2026. URL https://arxiv.org/abs/2604.16957.

[3] Oluwaseun A. Ajayi and Ogundepo Odunayo. Benchmarking On-Device Machine Learning on Apple Silicon with MLX. 2025. URL https://arxiv.org/abs/2510.18921.

[4] Willy Fitra Hendria. Non-Monotonic Latency in Apple MPS Decoding: KV Cache Interactions and Execution Regimes. 2026. URL https://arxiv.org/abs/2605.08913.

[5] Afsara Benazir and Felix Xiaozhu Lin. Efficient Mixture-of-Experts LLM Inference with Apple Silicon NPUs. 2026. URL https://arxiv.org/abs/2604.18788.

[6] Yoichi Ochiai. Systematic Optimization of Real-Time Diffusion Model Inference on Apple M3 Ultra. 2026. URL https://arxiv.org/abs/2605.16259.

[7] Jiedong Lang, Zhehao Guo, and Shuyu Huang. A Comprehensive Study on Quantization Techniques for Large Language Models. 2024. URL https://arxiv.org/abs/2411.02530.

[8] Jaewoo Song and Fangzhen Lin. SplitQuantV2: Enhancing Low-Bit Quantization of LLMs Without GPUs. 2025. URL https://arxiv.org/abs/2503.07657.

[9] Fangxin Liu, Ning Yang, Junping Zhao, Tao Yang, Haibing Guan, and Li Jiang. LCD: Advancing Extreme Low-Bit Clustering for Large Language Models via Knowledge Distillation. 2025. URL https://arxiv.org/abs/2506.12038.

[10] Sangjun Lee, Seung taek Woo, Jungyu Jin, Changhun Lee, and Eunhyeok Park. AMQ: Enabling AutoML for Mixed-precision Weight-Only Quantization of Large Language Models. 2025. URL https://arxiv.org/abs/2509.12019.

[11] Plawan Kumar Rath and Rahul Maliakkal. Weight Pruning Amplifies Bias: A Multi-Method Study of Compressed LLMs for Edge AI. 2026. URL https://arxiv.org/abs/2605.08137.

[12] Tuowei Wang, Ruwen Fan, Minxing Huang, Zixu Hao, Kun Li, Ting Cao, Youyou Lu, Yaoxue Zhang, and Ju Ren. Neuralink: Fast LLM Inference on Smartphones with Neuron Co-Activation Linking. 2024. URL https://arxiv.org/abs/2410.19274.

[13] Fucheng Jia, Zewen Wu, Shiqi Jiang, Huiqiang Jiang, Qianxi Zhang, Yuqing Yang, Yunxin Liu, Ju Ren, Deyu Zhang, and Ting Cao. Scaling Up On-Device LLMs via Active-Weight Swapping Between DRAM and Flash. 2025. URL https://arxiv.org/abs/2504.08378.

[14] Mingyu Sung, Vikas Palakonda, Suhwan Im, Sunghwan Moon, Il-Min Kim, Sangseok Yun, and Jae-Mo Kang. Memory- and Latency-Constrained Inference of Large Language Models via Adaptive Split Computing. 2025. URL https://arxiv.org/abs/2511.04002.

[15] Pablo Robin Guerrero, Yueyang Pan, and Sanidhya Kashyap. Efficient Deployment of Vision-Language Models on Mobile Devices: A Case Study on OnePlus 13R. 2025. URL https://arxiv.org/abs/2507.08505.

[16] Kunjal Panchal, Saayan Mitra, Somdeb Sarkhel, Haoliang Wang, Ishita Dasgupta, Gang Wu, and Hui Guan. Atom: Efficient On-Device Video-Language Pipelines Through Modular Reuse. 2025. URL https://arxiv.org/abs/2512.17108.